



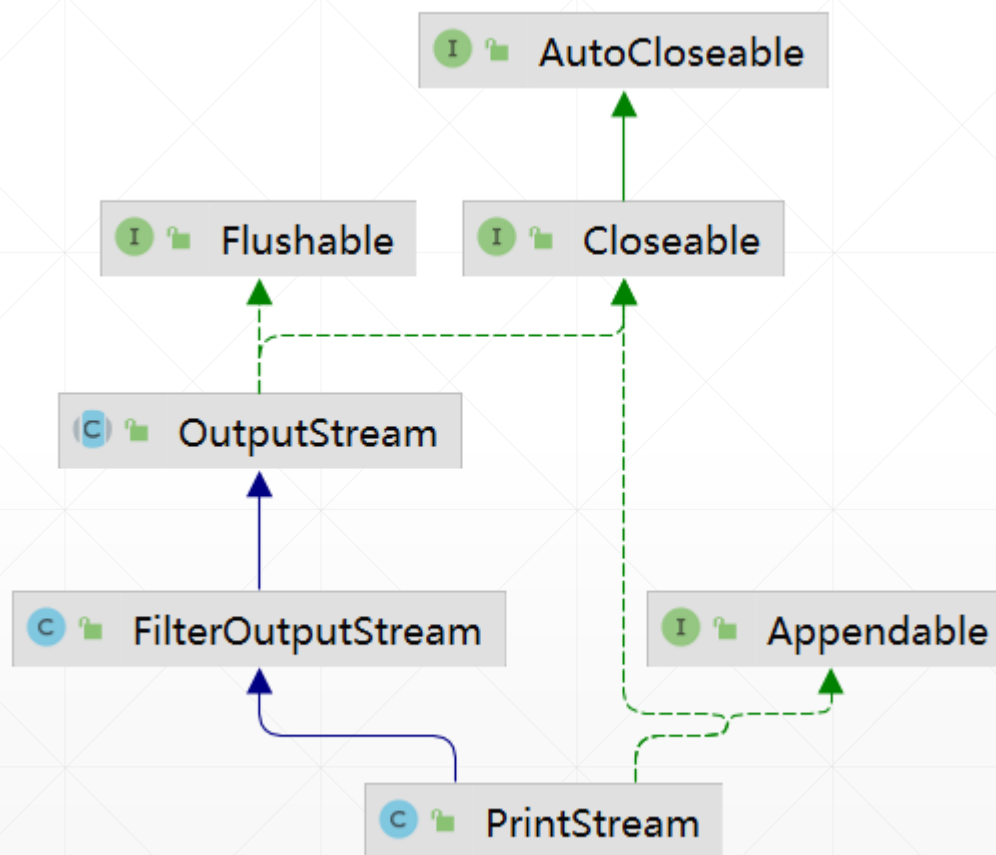
PrintStream与IO重定向

北京理工大学计算机学院
金旭亮

PrintStream概述

我们常用的System.out被称为“**标准输出流**”，它被定义为PrintStream类型的静态字段：

```
public static final PrintStream out = null;
```



PrintStream

PrintStream构造于OutputStream对象之上:

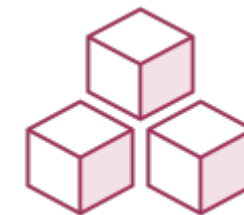
```
public PrintStream( OutputStream out )
```

PrintStream

OutputStream

主要重写了基类定义的诸如write()之类的几个方法，又定义了新的有很多重载形式的print/println/printf、append、format等几个方法。

PrintStream的使用示例



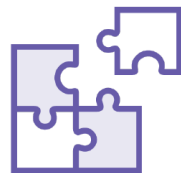
//基于输出流而构建

```
private static void basedOnOutputStream() {  
    PrintStream ps = null;  
    //创建一个文件输出流: FileOutputStream  
    try(FileOutputStream fos = new FileOutputStream(name: "poem.txt")) {  
        //以PrintStream来包装FileOutputStream输出流  
        ps = new PrintStream(fos);  
        //使用PrintStream执行输出  
        ps.println("打油诗一首");  
        ps.println("远看泰山黑乎乎");  
        ps.println("上边细来下边粗");  
        ps.println("有朝一日倒过来");  
        ps.println("下边细来上边粗");  
    } catch (IOException ioe) {  
        ioe.printStackTrace(ps);  
    }  
    System.out.println("数据已经写入poem.txt");  
}
```

对于这种典型场景，
可以进行简化.....

UsePrintStreamSaveText.java

简化后的PrintStream构建



直接将文件名传给PrintStream，就能直接向文件中写入数据。只要查看一下源代码就知道，其实里面还是new了一个FileOutputStream。

```
private static void directUse() throws FileNotFoundException {  
    //直接实例化PrintStream对象  
    try (var printStream = new PrintStream(fileName: "poem.txt")) {  
        printStream.println("打油诗一首");  
        printStream.println("远看泰山黑乎乎");  
        printStream.println("上边细来下边粗");  
        printStream.println("有朝一日倒过来");  
        printStream.println("下边细来上边粗");  
    };  
    System.out.println("数据已经写入poem.txt");  
}
```

UsePrintStreamSaveText.java

使用PrintStream复制文件



PrintStream本质上是一个输出流，可以用来接收外部输入的数据，所以，可以用来复制文件

```
public class UsePrintStreamToCopyFile {  
    public static void main(String[] args) {  
        String sourceImagePath = "images/myimage.jpg";  
        String targetImagePath = "images/myimage_copy.jpg";  
        try (var fileInputStream = new FileInputStream(sourceImagePath);  
            var printStream = new PrintStream(targetImagePath);  
        ) {  
            //直接将数据从一个流复制到另一个流  
            fileInputStream.transferTo(printStream);  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
        System.out.println("文件复制结束");  
    }  
}
```

UsePrintStreamToCopyFile.java

三种标准IO设备



System.in: 标准输入设备 (Standard Input) , 通常是键盘, InputStream 类型

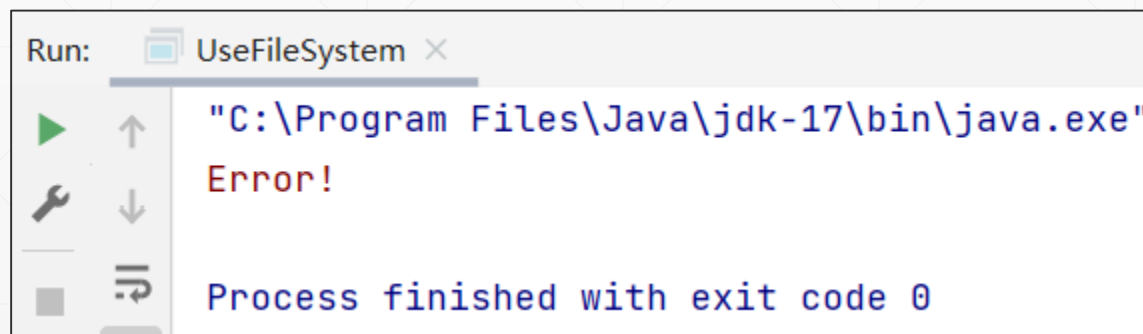


System.out: 标准输出 (Standard output) , 通常是控制台窗口, PrintStream 类型



System.err: 标准出错设备 (Standard error) , 默认情况下, 会在控制台窗口中以红色输出信息, PrintStream 类型

```
public static void main(String[] args) {  
    System.err.println("Error!");  
}
```



IO重定向



System类提供了以下方法，可以将标准IO重定向到某个流上：

- `void setIn(InputStream in)`
- `void setOut(PrintStream out)`
- `void setErr(PrintStream err)`

重定向输入

```
private static void redirectIn() {
    try (var fis = new FileInputStream(name: "src/RedirectIO.java")) {
        //将标准输入重定向到fis输入流
        System.setIn(fis);
        //使用System.in创建Scanner对象, 用于获取标准输入
        Scanner sc = new Scanner(System.in);
        //增加下面一行将只把回车作为分隔符
        sc.useDelimiter(pattern: "\\n");
        //判断是否还有下一个输入项
        while (sc.hasNext()) {
            //输出输入项
            System.out.println("从文件提取: " + sc.next());
        }
    } catch (IOException ex) {
        ex.printStackTrace();
    }
}
```

System.in原来指键盘，现在变成了文件，所以，文件中的内容就成了输入的数据。

```
RedirectIO {
    从文件提取:
    从文件提取: private static void redirectIn() {
    从文件提取:     try (var fis = new FileInputStream("./src/RedirectIO.java")) {
    从文件提取:         //将标准输入重定向到fis输入流
    从文件提取:         System.setIn(fis);
    从文件提取:         //使用System.in创建Scanner对象, 用于获取标准输入
    从文件提取:         Scanner sc = new Scanner(System.in);
    从文件提取:         //增加下面一行将只把回车作为分隔符
    从文件提取:         sc.useDelimiter("\n");
    从文件提取:         //判断是否还有下一个输入项
    从文件提取:         while (sc.hasNext()) {
    从文件提取:             //输出输入项
    从文件提取:             System.out.println("从文件提取: " + sc.next());
    从文件提取:         }
    从文件提取:     }
    从文件提取: }
}
```

重定向输出

```
private static void redirectOut() {  
    try (PrintStream ps = new PrintStream(  
        new FileOutputStream(name: "out.txt"))) {  
        //将标准输出重定向到ps输出流  
        System.setOut(ps);  
        //向标准输出输出一个字符串  
        System.out.println("普通字符串");  
        //向标准输出输出一个对象  
        System.out.println(new Date());  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

RedirectIO.java



out.txt	
1	普通字符串
2	Sat Sep 24 17:46:42 CST 2022
3	

重定向Err

```
private static void redirectErr() throws FileNotFoundException {  
    String LOGFILENAME = "runInfo.log";  
    try (var printStream = new PrintStream(new FileOutputStream(LOGFILENAME))) {  
        System.setErr(printStream);  
        System.out.println("运行时信息被写入文件: " + LOGFILENAME);  
        System.err.println("-----本程序的运行信息日志-----");  
        System.err.println("错误发生于: " + LocalTime.now());  
    };  
}
```

RedirectIO.java

